How often are the skill tags right? A check of 50 descriptions

1. The check

Ground truth. A skill is **required** if a name from the skill list appears in the ad outside a nice-to-have sentence. Sentences that say "Bonus if", "Nice to have" or "is a plus, not a must" mark **nice-to-have** skills, which are not required. The job title sentence is ignored (e.g. "Machine Learning Intern" does not ask for Machine Learning). **Tag right** = maps to a required skill. **Tag wrong** = a nice-to-have skill, a list skill the ad never mentions, or a tag not on the list. **Missed** = a required skill with no right tag. **Synonyms** count as right when they mean the same skill (Node, Postgres, Excel, JS, ML, API, "Financial Modeling", "Social Media") but I flag them, because a filter that matches on the exact list name would not see them. **Nice-to-have** skills are not required, so tagging one is wrong (a student who lacks it could be sent away or shown a role they can still do), and not tagging one is not a miss. All 50 ads were checked, no sample.

2. The numbers (200 tags, 177 required skills)

Measure	Count	Rate
Tags right (precision)	152 / 200	76.0%
Tags wrong	48 / 200	24.0%
nice-to-have tagged as required	20	10.0% of tags
list skill not in the text	12	6.0% of tags
not on the skill list	16	8.0% of tags
Required skills found (recall)	152 / 177	85.9%
Required skills missed	25 / 177	14.1%
Ads with no wrong tag and no miss	10 / 50	20.0%

Reading the range. If nice-to-have tags were accepted, precision is 172/200 = 86.0%. If variant names (18 tags) were counted wrong, it is 134/200 = 67.0%. So the honest answer is "about three tags in four", and one required skill in seven is lost. 35 of 50 ads carry at least one wrong tag; 20 miss at least one skill.

3. Three ways it fails

A. Tags the ad never supports (28 tags, 25 ads). The model adds filler: soft skills that are not on the list (Leadership 7, Teamwork 3, Problem Solving 3, Communication 2, Time Management 1) and unrelated list skills (SAP 4, Kubernetes 3, Blockchain 3, Machine Learning 2). Examples: **JD040** (HR intern given Leadership and SAP), **JD030** (Blockchain and Leadership), **JD025** (Kubernetes on a Marketing ad), **JD005** (Machine Learning on a Marketing ad).
B. Nice-to-have read as required (20 tags). The model tagged 20 of the 50 bonus skills (40%) whatever the wording: "Bonus" 9 of 20, "Nice to have" 7 of 15, "plus, not a must" 4 of 15. Examples: **JD009, JD039, JD049** (Zendesk, though the ad says Zendesk is a plus), **JD004** (SEO, a bonus), **JD029** (Customer Support, "nice to have", while the required Zendesk is missed).
C. Required skills dropped (25 skills, 20 ads). Misses are spread evenly over position in the list (first 16%, middle 10%, last 18%) and wording, so there is no simple trigger; ads with only 3 required skills lose 18%. 15 of the 20 ads pair a miss with a wrong tag, so a slot is spent on the wrong skill. A weak signal: in 4 cases the dropped skill is the job title (Machine Learning in **JD008** and **JD028**, Content Writing in **JD014**, Customer Support in **JD049**). Worst cases: **JD043** (PostgreSQL and REST APIs missing; only Node and Git tagged), **JD046** (Design Systems and Figma missing; Wireframing and Problem Solving added), **JD010** (HR Operations and Interviewing missing).

4. The fix I would make first

Make the model pick only from the skill list, and prove every tag. For each tag it must return the list skill, the exact phrase in the ad that asks for it, and a flag, required or nice-to-have. A cheap script then keeps a tag only if the phrase appears in the ad and the flag is "required". Why first: it removes pattern A completely (28 tags, 58% of all wrong tags, because Leadership or SAP cannot be quoted from the ad and soft skills are not on the list) and pattern B (20 tags) in one change, and it makes the 18 variant names disappear. **How I would check it:** rerun on these 50 ads and score with the same rules. Targets: wrong tags under 5% (now 24%), invented tags 0, nice-to-have tags near 0, and recall at least 85.9% so the fix does not trade wrong tags for missed skills. Then test on 50 new ads (ideally real ones, since these are made up and follow one template) so I am not tuning to this set, and read 20 rejected tags by hand to see whether good tags were thrown away. Pattern C needs a second step (for example, a pass that checks each list skill against the ad) and I would do it after this one.

Tool use: the labelling script (label.py) and the first draft of this write-up were produced with Claude. The rules above are what the script applies; each of the 50 ads was then read against its labels. Files: skill_tag_labels.xlsx (labels per ad and per tag, live formulas), label.py.